Image Compression Using Huffman Coding and Transform-Based 8×8 Block Processing: Design, Implementation and Performance Evaluation

RAM SYAM SUNDAR
Scholar, M.Tech
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
2601050102@kluniversity.in

Raju Anitha
Professor
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
anitharaju@kluniversity.in

***Abstract*—**Digital images dominate storage and network traffic, and efficient compression remains a core problem in multimedia systems. This paper presents the design, implementation and evaluation of a complete image compression and decompression tool built from two classical ideas: transform-based block processing and Huffman coding. The input image is converted to grayscale and divided into independent 8×8 blocks (divide and conquer). Each block is mapped to the frequency domain with the two-dimensional discrete cosine transform (DCT), quantized with a quality-controlled table, and serialized into a symbol stream using differential DC coding, zig-zag scanning and end-of-block markers. A Huffman tree is built with a min-heap (a greedy algorithm), and the stream is bit-packed into a self-describing binary file. The decompressor reverses each step. The system is implemented in Python with eight core classes, a command-line interface and unit tests. At quality 50 it achieves compression ratios of 26.6:1, 10.8:1 and 6.9:1 on a synthetic image and two natural photographs, with PSNR between 28.2 dB and 38.8 dB, whereas Huffman coding applied directly to pixels yields only 1.03:1 to 1.12:1. The Huffman stage operates within 0.02–0.06 bit/symbol of the source entropy; relative to the libjpeg encoder with optimized tables, the files are 4% to 25% larger at identical PSNR, which an analysis of the symbol stream attributes to the absence of run-length symbols. Measured running time grows linearly with image size, in agreement with the complexity analysis.

***Index Terms*—Image compression, Huffman coding, discrete cosine transform, quantization, block transform coding, greedy algorithms, divide and conquer, rate–distortion.**

I. INTRODUCTION

A. Background

A grayscale image of 512×512 pixels occupies 262,144 bytes when stored as raw 8-bit samples, and a photograph taken with a modern phone occupies tens of megabytes before compression. Storing and transmitting such data directly is wasteful because natural images contain large amounts of redundancy. Neighboring pixels are strongly correlated (*spatial redundancy*), the distribution of sample values is far from uniform (*coding redundancy*), and the human visual system is insensitive to some high-frequency detail (*psychovisual redundancy*) [16]. Image compression reduces the number of bits needed to represent an image by exploiting these three kinds of redundancy.

Practical lossy image coders, of which JPEG is the best-known example, follow a common pattern: decorrelate the data with a block transform, discard perceptually unimportant information by quantization, and entropy-code what remains [9], [10], [22]. Each of these stages rests on a classical result: the discrete cosine transform (DCT) approaches the performance of the optimal Karhunen–Loève transform for highly correlated sources [6], and Huffman's greedy algorithm produces the best possible prefix code for a given symbol distribution [2]. This paper presents the design, implementation and evaluation of a complete compressor and decompressor that follows this pattern end to end in a compact, readable Python code base.

The tool is also used as a vehicle to study two algorithm design paradigms [15]. *Divide and conquer* appears in the partitioning of the image into independent 8×8 blocks, which are transformed separately and merged afterwards. The *greedy method* appears in the construction of the Huffman tree, where the two least frequent nodes are merged repeatedly with the guarantee that the local choice is globally optimal.

B. Motivation

Compression algorithms are an excellent subject for study because they combine mathematics (transform theory, information theory), data structures (heaps, binary trees, bit streams) and software engineering (file formats, error handling, evaluation). A from-scratch implementation makes every design decision explicit and measurable, which is not possible when a production library such as libjpeg is used as a black box. A transparent implementation also provides a baseline against which improvements such as run-length symbols, canonical Huffman codes or arithmetic coding can be assessed quantitatively.

C. Problem Statement

Given an 8-bit image of arbitrary dimensions, design and implement a system that (a) produces a compressed binary file substantially smaller than the raw pixel data and (b) reconstructs from that file an image whose distortion with respect to the original is controllable and quantifiable. The solution must satisfy five requirements, labeled R1–R5 and traced to the implementation in Section IV:

1) **R1 (divide and conquer):** the image is partitioned into 8×8 blocks that are processed independently and merged afterwards.
2) **R2 (transform coding):** every block is transformed with the DCT, and the inverse DCT is used for reconstruction.
3) **R3 (greedy entropy coding):** the transformed data are encoded with a Huffman code built with a priority queue; frequency counting, tree construction, code generation, encoding and decoding are all implemented.
4) **R4 (object-oriented design):** the functionality is organized into the classes ImageProcessor, TransformBlock, HuffmanNode, HuffmanTree, Encoder,

Decoder, CompressionManager and DecompressionManager.

5) **R5 (usability and quality):** the tool runs from the command line, validates its inputs, and is accompanied by tests, documentation and a complexity analysis.

A subtle issue shapes the solution. The DCT of 8-bit data yields real-valued coefficients, and Huffman coding of real numbers that are almost all distinct gives no compression. A quantization step that maps the coefficients to a small set of integers is therefore an essential part of the pipeline even though it is not listed in the basic transform-then-encode workflow. Quantization is the only source of information loss in the system.

D. Contributions

The contribution of this work is not a new compression algorithm but a clean, tested and fully measured implementation of a well-understood architecture, together with an analysis of how much each modeling decision contributes. Specifically:

- a modular, object-oriented compressor/decompressor (eight core classes, a command-line interface and eight unit tests) with a self-describing, safely parsed container format;
- a quantitative ablation of the symbol model showing the separate contributions of the transform, zig-zag scanning with end-of-block markers, and differential DC coding (Section VII-D);
- an entropy-efficiency and bit-allocation analysis showing that the Huffman stage operates within 0.02–0.06 bit/symbol of the source entropy, and that redundant "interior" zero symbols are the main source of inefficiency relative to JPEG (Sections VII-E and VII-F);
- a head-to-head comparison with the libjpeg encoder with optimized Huffman tables at identical quantization, isolating the entropy-coding stage (Section VII-G); and
- an empirical confirmation that running time scales linearly with image size, matching the complexity analysis (Sections VII-H and VIII).

E. Organization of the Paper

Section II surveys related work. Section III summarizes the theory behind entropy coding, the DCT and quantization. Section IV describes the system architecture, class design and file format, and Section V the algorithms in detail. Section VI covers the implementation and testing. Section VII reports the experimental results, Section VIII gives the complexity analysis, Section IX discusses advantages, limitations and threats to validity, Section X outlines future work, and Section XI concludes. The appendices contain the user guide and the main source listings.

II. RELATED WORK

A. Entropy and Huffman Coding

Shannon showed that the average number of bits per symbol needed to represent a memoryless source cannot fall below its entropy [1], [21]. Huffman's 1952 algorithm constructs a prefix code that is optimal among all codes assigning an integer number of bits to each symbol; its average length L satisfies $H \le L < H + 1$ [2]. The algorithm is greedy: it repeatedly merges the two least probable symbols, and a standard exchange argument proves that this local choice leads to a globally optimal code [15]. Gallager analyzed the redundancy of Huffman codes and the structure of the resulting trees [3], and Schwartz and Kallick introduced canonical codes, which allow a decoder to reconstruct a code from code lengths alone [4].

B. Arithmetic Coding

Arithmetic coding represents an entire message as a sub-interval of [0, 1) and can approach the entropy within a very small overhead even when symbol probabilities are not powers of two [5]. It also adapts naturally to changing statistics. Its cost is greater implementation complexity, and historically patent concerns slowed its adoption in still-image standards. Context-adaptive binary arithmetic coding was later adopted in video standards such as H.264/AVC [14]. Huffman coding remains attractive for educational and lightweight implementations because of its simplicity and speed.

C. Transform Coding and the DCT

Transform coding maps a block of correlated samples to less correlated coefficients so that most of the signal energy is concentrated in a few of them. The optimal linear transform for a Gaussian source is the Karhunen–Loève transform, but it depends on the signal statistics and has no fast algorithm. Ahmed, Natarajan and Rao introduced the DCT and showed that on first-order Markov data with high correlation its performance closely approaches that of the Karhunen–Loève transform [6]. The DCT is real-valued, separable, has an orthonormal form and admits fast algorithms such as that of Loeffler, Ligtenberg and Moschytz, which computes the 8-point 1-D transform with 11 multiplications [7]. Rao and Yip give a comprehensive treatment [8]. Compared with the discrete Fourier transform, the DCT implicitly uses a symmetric extension of the block, which avoids the boundary discontinuities responsible for spectral leakage and makes it better suited to compression.

D. The JPEG Standard

Baseline JPEG divides the image into 8×8 blocks, applies the DCT, quantizes the coefficients with a perceptually tuned table, orders them in zig-zag fashion, codes the DC coefficients differentially, run-length codes the zero runs of the AC coefficients, and entropy-codes the resulting symbols with Huffman tables [9], [10], [22]. The quantization tables are scaled by a quality factor, the same mechanism adopted in this work. The principal difference between JPEG and the present implementation lies in the symbol model: JPEG forms joint (zero-run, magnitude-category) symbols followed by raw magnitude bits, whereas this tool Huffman-codes the coefficient values themselves and relies on end-of-block markers to remove trailing zeros.

E. Wavelet-Based Compression

JPEG 2000 replaces the block DCT by a wavelet transform applied to the whole image or to large tiles, followed by embedded block coding with optimized truncation and arithmetic coding [11], [12]. It avoids the blocking artifacts typical of low-bit-rate DCT coding and supports scalability, at considerably higher complexity. For a project that must demonstrate block-wise divide and conquer, the block DCT is the natural choice.

F. Image Quality Measurement

Mean squared error (MSE) and peak signal-to-noise ratio (PSNR) are the most common objective measures of reconstruction fidelity. They are simple and cheap to compute but correlate only loosely with perceived quality. Structural measures such as SSIM compare local luminance, contrast and structure [13]. This work reports MSE and PSNR; SSIM is left as an extension.

G. Summary and Positioning

TABLE I
COMPARISON OF RELATED TECHNIQUES

Technique	Core idea	Strength	Weakness
Huffman coding	Optimal integer-length prefix code	Simple, fast, optimal in its class	Cannot exploit pixel correlation alone
Arithmetic coding	Interval subdivision	Near-entropy rate; adaptive	More complex to implement
Block DCT (JPEG)	Decorrelate 8×8 blocks	Energy compaction; fast	Blocking at low rates
Wavelet (JPEG 2000)	Multi-resolution transform	No blocking; scalable	Higher complexity
This work	Block DCT + quantization + Huffman	Transparent, modular, measurable	No run-length symbols; grayscale only

The literature shows that the combination of a block DCT, quantization and Huffman coding is a well-understood and effective architecture. The gap this work addresses is practical rather than theoretical: few publicly documented implementations expose every stage, quantify what each modeling decision contributes, and compare against a production encoder under controlled conditions.

III. THEORETICAL BACKGROUND

A. Entropy and Prefix Codes

Let a source emit symbols from an alphabet of n distinct values with probabilities p_i. The entropy is

$$H = -\Sigma_{i=1}^{n} p_i \log_2 p_i \quad \text{[bit/symbol]} \tag{1}$$

A code is a *prefix code* if no code word is a prefix of another; such a code can be decoded instantaneously by walking a binary tree from the root and emitting a symbol at each leaf. The lengths l_i of any binary prefix code satisfy the Kraft inequality, $\Sigma\, 2^{-l} \le 1$, where the sum runs over the code words and l is the length of a code word; conversely, any set of lengths satisfying it is realizable [21]. The average length is $L = \Sigma\, p_i\, l_i$, and the source coding theorem yields

$$H \le L < H + 1. \tag{2}$$

The upper bound is the guarantee provided by Huffman coding. Its redundancy $L - H$ is largest when one symbol has probability close to one, because an integer-length code cannot spend less than one bit on it. Section VII-E measures this redundancy for the compressor's actual symbol streams.

B. Optimality of the Greedy Construction

Huffman's algorithm repeatedly removes the two nodes of smallest weight, makes them the children of a new node whose weight is their sum, and reinserts the new node. Its optimality rests on two properties [15]. *Greedy-choice property:* if x and y are the two least frequent symbols, there is an optimal prefix tree in which x and y are sibling leaves at maximum depth. The proof exchanges x and y with the two deepest sibling leaves of any optimal tree; the exchange cannot increase the cost, because moving a less frequent symbol deeper and a more frequent symbol shallower never increases $\Sigma\, f_i\, l_i$. *Optimal substructure:* if the merged node z is treated as a single symbol of weight $f_x + f_y$, the cost of a tree for the original alphabet equals the cost of the corresponding tree for the reduced alphabet plus $f_x + f_y$, so an optimal tree for the reduced alphabet yields an optimal tree for the original. Induction on the alphabet size completes the proof.

C. The Two-Dimensional DCT

For an N-point sequence the orthonormal DCT-II is

$$X_k = c_k \sqrt{(2/N)}\, \Sigma_{m=0}^{N-1} x_m \cos[(2m+1)k\pi / 2N] \tag{3}$$

with $c_0 = 1/\sqrt{2}$ and $c_k = 1$ otherwise. Writing the transform as an $N \times N$ matrix T with entries $T_{km} = c_k \sqrt{(2/N)} \cos[(2m+1)k\pi/2N]$, T is orthogonal ($T\,T^{\mathrm{T}} = I$). For $N = 8$ the 2-D transform of a block $f(x, y)$ is separable:

$$F = T\, f\, T^{\mathrm{T}}, \quad f = T^{\mathrm{T}}\, F\, T \tag{4}$$

which in scalar form reads $F(u, v) = \tfrac{1}{4}\, c(u)\, c(v)\, \Sigma_x\, \Sigma_y\, f(x, y) \cos[(2x+1)u\pi/16] \cos[(2y+1)v\pi/16]$. The coefficient $F(0, 0)$ is the *DC* coefficient and equals eight times the block mean; the remaining 63 are *AC* coefficients that represent increasing horizontal and vertical spatial frequencies. Because T is orthogonal, the transform preserves energy (Parseval's relation): $\Sigma\, f^2 = \Sigma\, F^2$. Consequently an error introduced in the coefficient domain has exactly the same energy in the pixel domain, which is what allows quantization error to be reasoned about, and controlled, coefficient by coefficient.

D. Quantization

Quantization maps each coefficient to an integer index $F_q(u, v) = \mathrm{round}(F(u, v)/Q(u, v))$, where Q is a table of step sizes. The decoder reconstructs $\hat{F} = F_q\, Q$. Coefficients smaller than half a step size become zero (a dead zone around zero that is wide relative to the data), and the error of a coefficient that is not zeroed is approximately uniform on $[-Q/2, Q/2]$, with variance $Q^2/12$. By Parseval's relation the expected per-pixel squared error is approximately

$$\mathrm{MSE} \approx (1/64)\, \Sigma_{u,v}\, D(u, v) \tag{5}$$

where $D(u, v)$ is $Q^2/12$ for coefficients that survive and the coefficient energy itself for coefficients that are zeroed. Larger step sizes at high frequencies thus trade a small perceptual loss for a large reduction in the number of non-zero symbols. Quantization is the only irreversible step of the pipeline; rounding and clipping of the reconstructed pixels adds a further, much smaller error.

E. Statistical Model of Quantized Coefficients

The effectiveness of entropy coding after quantization depends on the shape of the coefficient histogram. The AC coefficients of natural images are well modeled by a zero-mean Laplacian density $f(x) = (\lambda/2)\, e^{-\lambda|x|}$ [16]. Quantizing such a coefficient with a uniform step Δ and rounding to the nearest integer gives the discrete distribution

$$P(0) = 1 - e^{-\lambda\Delta/2}, \quad P(\pm k) = \tfrac{1}{2}\, e^{-\lambda(k-\frac{1}{2})\Delta}\,(1 - e^{-\lambda\Delta}), \quad k \ge 1 \tag{6}$$

which is geometric in k apart from the central bin. When $\lambda\Delta$ is large (a coarse step relative to the coefficient scale $1/\lambda$), almost all of the probability mass sits at zero and the entropy of the quantized coefficient is a small fraction of a bit; as Δ shrinks the distribution spreads and the entropy grows by about one bit for every halving of the step size. Three consequences are relevant to the design. First, high-frequency coefficients, which have a small scale $1/\lambda$ and are also divided by large steps $Q(u, v)$, are zero with very high probability, which justifies replacing their tail by one end-of-block symbol. Second, the zero-order histogram of the pooled coefficients is sharply peaked and therefore compressible by a Huffman code, in contrast with the nearly flat histogram of raw pixel values. Third, because $P(0)$ can approach one, the integer-length limitation of Huffman coding (at least one bit per coded symbol) can become visible at very low rates; this is the situation in which arithmetic coding [5] gains most, and the reason the end-of-block marker is preferable to coding every individual zero.

F. Rate and Distortion Measures

The *compression ratio* is CR = B_{raw} / B_{comp}, where $B_{raw} = H \cdot W$ is the size of the raw 8-bit grayscale image and B_{comp} is the size of the complete compressed file, including header and frequency table. The *bit rate* is $R = 8\, B_{comp} / (H \cdot W)$ bits per pixel (bpp). Distortion is measured by

$$\mathrm{MSE} = (1/HW)\, \Sigma\, (I - \hat{I})^2, \quad \mathrm{PSNR} = 10 \log_{10}(255^2 / \mathrm{MSE}) \ \mathrm{dB} \tag{7}$$

with PSNR taken as infinite for identical images. The pairs (R, PSNR) over a range of quality settings form the *rate–distortion curve* of the codec and are the basis of all comparisons in Section VII.

G. Divide and Conquer in Block Coding

The block partition is a divide-and-conquer decomposition in the classical sense. *Divide:* split the (padded) image into $B = (H'/8)(W'/8)$ blocks. *Conquer:* transform and quantize every block independently, a sub-problem of constant size. *Combine:* merge the reconstructed blocks in raster order and crop the padding. The sub-problems do not interact, so the conquer step is embarrassingly parallel and costs O(1) per block, and the total transform cost is linear in the number of pixels. The price of independence is that correlation across block boundaries is not exploited, which causes the blocking artifacts discussed in Section VII-I.

IV. SYSTEM DESIGN

A. Design Goals and Requirements Traceability

The design goals are modularity (every stage is a separate, testable class), determinism (the decoder must rebuild exactly the structures the encoder used), self-description (the output file carries everything needed to decode it), robustness (malformed input must fail loudly, never silently produce a wrong image) and measurability (every stage exposes statistics). Table II traces each requirement of Section I-C to the module that satisfies it and the test or experiment that verifies it.

TABLE II
REQUIREMENTS TRACEABILITY

Req.	Implementing module	Key mechanism	Verification
R1	image_processor.py	Edge-padded reshape into 8×8 blocks	test_divide_and_merge; odd-size test
R2	transform_block.py	scipy.fft.dctn / idctn, orthonormal	test_dct_is_invertible
R3	huffman_tree.py, encoder.py, decoder.py	Min-heap tree; bit packing	prefix-free and round-trip tests
R4	eight core classes + utils	One responsibility per class	Section IV-C; code review
R5	main.py, tests/, docs/	argparse CLI; error codes; reports	corrupt-file test; Section VII

B. Layered Architecture

The system has four layers (Fig. 1). The *presentation layer* is a command-line program offering the commands compress, decompress and evaluate. The *orchestration layer* consists of CompressionManager and DecompressionManager, which call the processing classes in the correct order and handle files. The *processing layer* contains ImageProcessor, TransformBlock, Encoder and Decoder. The lowest layer holds the core data structures, HuffmanTree and HuffmanNode, together with a utility module for the file container, the zig-zag order and the quality metrics. Dependencies point only downward, so no component depends on a layer above it, and each class can be tested in isolation.

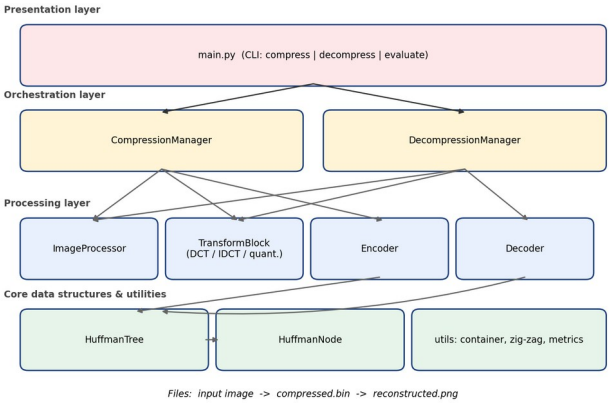


Fig. 1. Layered architecture of the image compression tool. Data flows input image → compressed.bin → reconstructed image.

C. Class Design

Fig. 2 shows the class diagram and Table III lists the responsibility of each class. Every class has exactly one responsibility. The managers hold no algorithmic logic; they instantiate the processing classes and connect their inputs and outputs, which keeps the compression and decompression pipelines readable as a short sequence of calls.

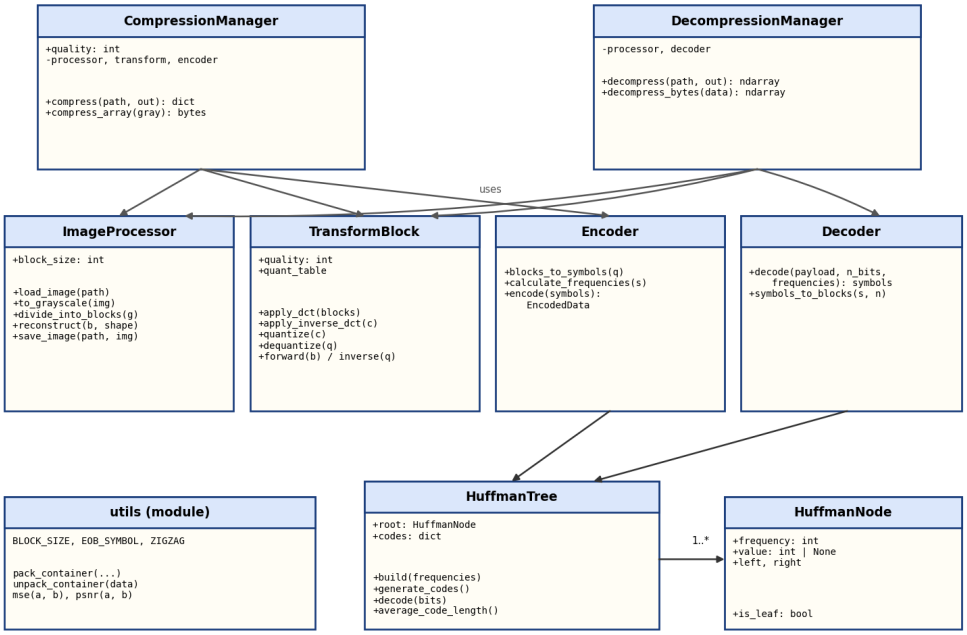


Fig. 2. UML-style class diagram. The two managers depend on the processing classes; Encoder and Decoder share HuffmanTree, which is composed of HuffmanNode objects.

TABLE III
CLASSES AND RESPONSIBILITIES

Class	Responsibility
ImageProcessor	Loads images, converts to 8-bit grayscale, pads and divides into 8×8 blocks, merges blocks and crops padding, saves images.
TransformBlock	Forward and inverse 2-D DCT; quantization and dequantization controlled by the quality factor.
HuffmanNode	Tree node holding frequency, value, left child and right child (uses __slots__).
HuffmanTree	Builds the tree with a min-heap, generates the codes, decodes bit sequences.
Encoder	Builds the symbol stream, counts frequencies, Huffman-encodes and bit-packs.
Decoder	Rebuilds the tree from frequencies, decodes bits and rebuilds quantized blocks.
CompressionManager	Runs the compression pipeline, writes the .bin file, returns statistics.
DecompressionManager	Reads the .bin file, runs the decompression pipeline, writes the image.

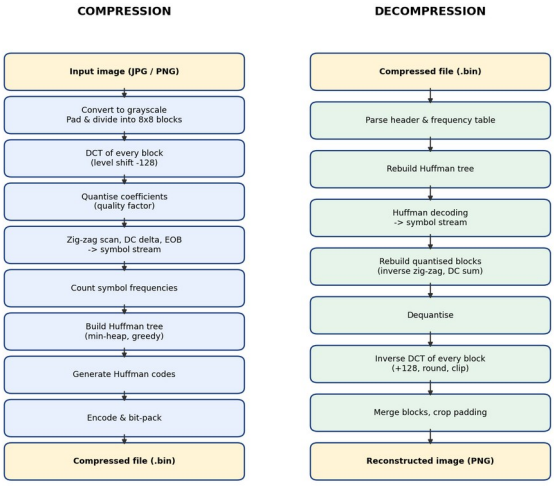


Fig. 3. Compression and decompression workflow.

D. Data Flow

Fig. 3 shows the two pipelines. Compression proceeds from the input image to the .bin file in nine steps: grayscale conversion and padding, block division, DCT with level shift, quantization, symbol-stream construction (zig-zag, differential DC, end-of-block marker), frequency counting, Huffman tree construction with code generation, and bit packing with container writing. Decompression retraces the steps in reverse order and ends with the merge of the blocks and the removal of the padding.

E. Compressed File Format

The .bin container (Table IV) consists of a 26-byte header, a frequency table of 8 bytes per distinct symbol, and the Huffman payload. Storing the frequency table rather than the code words keeps the decoder simple: it calls the same deterministic tree-building routine as the encoder and therefore obtains an identical tree. Metadata are serialized with Python's struct module rather than pickle, because unpickling data from an untrusted file can execute arbitrary code, and a fixed binary layout is also smaller and language-neutral.

TABLE IV
LAYOUT OF THE COMPRESSED FILE (LITTLE-ENDIAN)

Field	Size	Description
Magic	4 B	ASCII “HIMG”; identifies the format
Version	1 B	Container version (currently 1)
Height, width	4 + 4 B	Original image dimensions in pixels
Quality	1 B	Quality factor 1–100; selects the quantization table
Symbol count	4 B	Number n of entries in the frequency table
Bit count	8 B	Number of valid bits in the payload
Frequency table	8n B	n pairs (int32 symbol, uint32 frequency)
Payload	variable	Huffman bit stream, MSB first, zero-padded to a byte

As a check of the format, Table V decodes the 26-byte header of the file produced for the sample image at quality 50 (output/compressed.bin, 9,859 bytes). The stored bit count, 71,809, corresponds to a payload of 8,977 bytes; together with the header and the 107-entry frequency table (856 bytes) this accounts for the whole file: 26 + 856 + 8,977 = 9,859.

TABLE V
DECODED HEADER OF THE SAMPLE COMPRESSED FILE (Q = 50)

Field	Bytes (hex)	Value
Magic	48 49 4D 47	“HIMG”
Version	01	1
Height	00 02 00 00	512
Width	00 02 00 00	512
Quality	32	50
Symbol count	6B 00 00 00	107
Bit count	81 18 01 00 00 00 00 00	71,809
First table entry	C5 FF FF FF 02 00 00 00	symbol −59, frequency 2

F. Error Handling

The loader rejects missing files and undecodable images. The container parser verifies the magic number, the version, the table length and the payload length. The symbol-stream parser checks for missing end-of-block markers, blocks with more than 63 AC values, exhausted streams and trailing symbols. The quality factor is validated to lie in [1, 100]. Each violation raises an explicit exception (`ValueError` or `FileNotFoundError`) that the command-line program converts into a one-line message and a non-zero exit status.

V. ALGORITHMS

A. Image Partitioning

Let the grayscale image have H rows and W columns. The padded dimensions are $H' = 8\lceil H/8 \rceil$ and $W' = 8\lceil W/8 \rceil$; the missing rows and columns are filled by replicating the nearest edge pixel, which keeps the padded blocks smooth and therefore cheap to code. The padded image is divided into $B = (H'/8)(W'/8)$ blocks in raster order. In the implementation the division is a single NumPy reshape of the padded array to shape $(H'/8, 8, W'/8, 8)$ followed by an axis swap, so no Python loop over blocks is needed; the inverse reshape merges them. Inputs that are not 8-bit grayscale are normalized first: color images are converted to luminance, 16-bit images are rescaled to [0, 255] and alpha channels are discarded.

B. Forward and Inverse DCT

After subtracting 128 from each sample to center the data at zero, every block is transformed with the orthonormal 2-D DCT-II of Section III-C. The implementation calls scipy.fft.dctn with norm=“ortho” over the last two axes of the whole $(B, 8, 8)$ array, so that all blocks are processed by one vectorized call. Reconstruction applies idctn, adds 128, rounds to the nearest integer and clips to [0, 255].

C. Quantization

The quantization table is the standard JPEG luminance table [22], which defines the step sizes at quality 50, scaled by a factor s that depends on the quality $q \in [1, 100]$:

$$s = 5000/q \ (q < 50); \quad s = 200 - 2q \ (q \geq 50) \tag{8}$$

$$Q = \mathrm{clip}(\lfloor (Q_{50}\, s + 50) / 100 \rfloor, 1, 255) \tag{9}$$

Table VI shows how the step sizes of the lowest (DC) and highest (7, 7) frequencies change with quality. At $q = 100$ all steps equal 1, so only the rounding of coefficients to integers remains; at $q = 10$ the highest frequencies are divided by 255 and are almost always zeroed. Large divisors at high frequencies map small coefficients to zero, which is where most of the compression is obtained and where the loss occurs.

TABLE VI
QUANTIZATION STEP SIZES VERSUS QUALITY

Quality q	Scale s	$Q(0, 0)$	$Q(7, 7)$
10	500	80	255
25	200	32	198
50	100	16	99
75	50	8	50
90	20	3	20
100	0	1	1

D. Symbol Stream Construction

The quantized blocks are serialized into a stream of integer symbols. For each block in raster order: (1) the DC coefficient is emitted as a *difference* from the previous block’s DC; (2) the 63 AC coefficients are read in *zig-zag order*, which places low frequencies first and zeros last, and are emitted up to the last non-zero value; (3) a special *end-of-block* symbol EOB = 32768 is emitted. The value 32768 lies far outside the range of any quantized coefficient or DC difference of an 8-bit image (a DC difference is at most about ±2,048), so it cannot collide with data. The decoder inverts the process: it accumulates the DC differences, reads values until EOB, fills the rest of the block with zeros and undoes the zig-zag permutation.

Zig-zag scanning orders the coefficients by increasing frequency index $u + v$, alternating the direction along each anti-diagonal, so that the probable zeros form one contiguous tail that the single EOB symbol replaces. Differential coding of the DC term exploits the similarity of the mean intensity of neighboring blocks: the difference symbols are concentrated near zero and are cheaper to code than the absolute values.

Worked example.

Table VII and Table VIII show one 8×8 block of the sample image (block row 20, column 30) before and after the DCT and quantization at $q = 50$. The orthonormal DCT of the level-shifted block has $F(0, 0) = -252.0$, which the step size 16 maps to $-15.75 \rightarrow -16$. Of the 64 quantized coefficients only 10 are non-zero. In zig-zag order the sequence is

```
-16, 7, 1, 0, 1, 6, 3, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, ...0
```

with the last non-zero value at zig-zag position 15. The block is therefore represented by one DC-difference symbol, 15 AC symbols (six of them interior zeros) and one EOB symbol, 17 symbols instead of 64. Reconstruction from this block gives a per-pixel MSE of 18.0, larger than the 8.6 of the whole image because the block lies on a strong edge of the image.

TABLE VII
PIXEL VALUES OF THE EXAMPLE BLOCK

y\x	0	1	2	3	4	5	6	7
0	152	111	89	89	88	90	90	90
1	141	106	91	94	93	93	90	89
2	134	101	91	92	90	89	89	90
3	132	98	90	89	87	87	89	92
4	128	95	91	90	91	91	90	92
5	128	94	92	89	91	91	89	90
6	129	94	92	86	89	89	86	87
7	126	93	93	88	91	90	87	88

TABLE VIII
QUANTIZED COEFFICIENTS OF THE EXAMPLE BLOCK (Q = 50)

*v**u*	0	1	2	3	4	5	6	7
0	−16	7	6	3	1	1	0	0
1	1	1	1	0	0	0	0	0
2	0	1	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0

E. Huffman Coding

Frequency calculation.

A hash table (collections.Counter) counts the occurrences of each distinct symbol of the stream in O(*N*) time.

Tree construction with a min-heap.

Each distinct symbol becomes a leaf node keyed by (frequency, insertion order). The two nodes with the smallest keys are repeatedly removed from the heap and joined under a new internal node whose frequency is their sum, and the new node is pushed back (Algorithm 1). The process stops when one node, the root, remains. The insertion order in the key makes tie-breaking deterministic, which is what allows the decoder to rebuild an identical tree from the frequency table alone.

Algorithm 1. Huffman tree construction (greedy)

```
BUILD-HUFFMAN(freq)
 heap <- empty min-heap; order <- 0
 for each (symbol, f) in sorted(freq):
   push (f, order, Leaf(symbol)); order++
 while |heap| > 1:
   (f1, _, a) <- pop-min(heap)
   (f2, _, b) <- pop-min(heap)
   push (f1+f2, order, Node(a, b)); order++
 return the only node left in heap
```

Code generation.

A depth-first traversal labels each left edge 0 and each right edge 1; the path from the root to a leaf is that symbol's code word. The traversal uses an explicit stack, because the tree can be as deep as $n - 1$ for skewed distributions and Python's recursion limit would otherwise be a risk. If only one distinct symbol exists, it receives the code word 0 so that the stream is not empty. Because the code is prefix-free, decoding is unambiguous.

A small worked example.

For the frequency table *A*:45, *B*:13, *C*:12, *D*:16, *E*:9, *F*:5 (a standard textbook example [15]) the implementation produces the codes in Table IX. The average length is 2.24 bit/symbol against an entropy of 2.22 bit/symbol and a fixed-length cost of 3 bit/symbol, a saving of 25%. The frequent symbol *A* receives a one-bit code and the rare symbols *E* and *F* four-bit codes, illustrating how the greedy merge order pushes rare symbols deep into the tree.

TABLE IX
HUFFMAN CODES FOR THE EXAMPLE ALPHABET

Symbol	Freq.	Code	Length	Fixed (3 b)
A	45	0	1	3
D	16	111	3	3
B	13	101	3	3
C	12	100	3	3
E	9	1101	4	3
F	5	1100	4	3

Encoding and decoding.

Encoding concatenates the code words of all symbols and packs the resulting bit string into bytes with numpy.packbits; the number of valid bits is stored in the header. Decoding unpacks the bytes, truncates them to the stored bit count and walks the tree from the root, taking the left child on a 0 bit and the right child on a 1 bit, emitting a symbol and returning to the root whenever a leaf is reached. A stream that ends inside a code word is reported as corrupt.

F. Determinism and Correctness Argument

The decoder never receives the code words; it receives the frequency table and must reconstruct the same tree. This works because the construction is a deterministic function of the table. Leaves are inserted in sorted symbol order and every heap key is the pair (frequency, insertion counter); since the counter is unique, the keys form a strict total order and the sequence of pop-min operations is uniquely determined. The encoder and decoder therefore build identical trees and assign identical code words, and the entropy-coding stage is exactly invertible: symbols map to bit strings through a prefix code, and bit strings map back to symbols by tree traversal.

The remaining stages are also invertible except for quantization. Block division and merging are inverse permutations of the pixels (after cropping the padding); the DCT is an orthogonal transform and thus invertible; the zig-zag permutation, the DC differencing and the end-of-block terminator define an invertible mapping between a quantized block and its symbol sequence, provided the EOB value cannot occur as data, which the choice EOB = 32768 guarantees. The only information lost in the whole pipeline is therefore the rounding of coefficients in the quantizer, together with the small rounding and clipping error of the final inverse transform. This is also why the error of the codec can be controlled with a single parameter.

G. Special Cases

- **Image size not a multiple of 8:** edge padding at the encoder, cropping at the decoder; the header stores the true dimensions.
- **Constant image:** all AC coefficients vanish, every DC difference after the first is zero, and the symbol alphabet reduces to a handful of values; if only one distinct symbol remained, it would receive the code word "0". The test suite verifies exact reproduction.
- **Very high quality:** at $q = 100$ all step sizes are 1 and the alphabet grows to several hundred symbols, increasing the table overhead (Section VII-E).
- **Very low quality:** step sizes reach 255, nearly all AC coefficients become zero, and the stream is dominated by DC differences and end-of-block markers (Section VII-F).

H. Complete Pipelines

Algorithm 2. Compression

```
COMPRESS(image, q)
 gray   <- GRAYSCALE(image)
```

```
blocks <- DIVIDE(PAD(gray))          // divide
coeffs <- DCT(blocks - 128)          // conquer
quant  <- ROUND(coeffs / Q(q))
syms   <- []
for each block b in raster order:
  syms += [DC(b) - DC(prev)]
  syms += ZIGZAG(b)[1 .. last nonzero]
  syms += [EOB]
freq   <- COUNT(syms)
tree   <- BUILD-HUFFMAN(freq)
bits   <- CONCAT(code[s] for s in syms)
return HEADER + TABLE(freq) + PACK(bits)
```

Algorithm 3. Decompression

```
DECOMPRESS(file)
 (H, W, q, freq, bits) <- PARSE(file)   // validate
 tree  <- BUILD-HUFFMAN(freq)
 syms  <- WALK(tree, bits)
 quant <- REBUILD-BLOCKS(syms)  // un-zigzag, sum DC
 coeffs <- quant * Q(q)
 blocks <- CLIP(ROUND(IDCT(coeffs) + 128))
 return CROP(MERGE(blocks), H, W)            // combine
```

VI. IMPLEMENTATION

A. Environment and Libraries

The tool is written in Python 3.11 or newer and was developed and tested under Python 3.12. NumPy [17] provides array handling and bit packing, SciPy [18] the DCT, OpenCV [20] image input/output and color conversion, and Matplotlib [19] the evaluation figures. The standard library supplies heapq for the priority queue and struct for the file container. The code follows PEP 8, uses type hints and docstrings, and has no dependencies beyond the four packages listed in requirements.txt.

B. Project Structure

```
ImageCompressionTool/
  src/    image_processor.py  transform_block.py
          huffman_node.py     huffman_tree.py
          encoder.py          decoder.py
          compression_manager.py
          decompression_manager.py
          main.py  utils.py  evaluation.py
  input/  sample_image.jpg
  output/ compressed.bin, reconstructed.png, figures
  docs/   design, algorithm and complexity notes,
          report, images/
  tests/  test_roundtrip.py
  scripts/ sample-image and diagram generators
  requirements.txt  setup.cfg  README.md  LICENSE
```

C. Key Code Excerpts

The pixel-to-coefficient stage processes all blocks at once by operating on the last two axes (Listing 1). Because no Python loop runs over blocks, the transform of a 2048×2048 image (65,536 blocks) takes a fraction of a second.

```
shifted = blocks.astype(np.float64) - 128.0
coeffs = dctn(shifted, type=2, norm="ortho",
              axes=(-2, -1))
quantized = np.rint(coeffs / self.quant_table)
              .astype(np.int32)
```

Listing 1. Vectorized forward transform and quantization (TransformBlock).

The symbol stream is produced by Encoder.blocks_to_symbols (Listing 2). The zig-zag permutation is applied to all blocks in one fancy-indexing operation; only the final assembly of the stream loops over blocks.

```
zz = quantized.reshape(n_blocks, -1)[:, ZIGZAG]
previous_dc = 0
for row in zz:
    dc = int(row[0])
    symbols.append(dc - previous_dc)
    previous_dc = dc
    nonzero = np.flatnonzero(row[1:])
    if nonzero.size:
        symbols.extend(row[1:nonzero[-1] + 2].tolist())
    symbols.append(EOB_SYMBOL)
```

Listing 2. Symbol-stream construction (Encoder).

The Huffman stage itself (heap-based construction, iterative code generation and bit-level decoding) is shown in full in Appendix B, together with the remaining modules.

D. Command-Line Interface

The CLI is implemented with argparse and offers three commands. compress takes an input image, an optional output path and a quality factor (default 50) and prints the image size, raw size, compressed size, compression ratio and bits per pixel. decompress takes a .bin file and optionally the original image, in which case it prints MSE and PSNR. evaluate sweeps the quality factor, prints a results table and writes the rate–distortion curve and a visual comparison to the output directory. All errors are reported as a single message with exit status 1. Appendix A gives complete usage examples.

E. Testing

The test suite (unittest, eight tests) covers the properties on which correctness depends (Table X). All tests pass in about 0.02 s. In addition to the unit tests, the end-to-end experiments of Section VII act as integration tests: every compressed file in the evaluation was decoded and its PSNR was measured, and the PSNR values agreed to within 0.01 dB with the reference JPEG encoder at the same quantization.

TABLE X
UNIT TESTS

Test	What it verifies
prefix_free_and_roundtrip	No code word is a prefix of another; decoding returns the original symbols.
single_symbol	A one-symbol alphabet receives code “0” and decodes correctly.
dct_is_invertible	DCT, then inverse DCT, reproduces random 8-bit blocks exactly after rounding.
divide_and_merge	Block division and merging are inverse for sizes not divisible by 8.
odd_size_image	Full pipeline round trip on an odd-sized random image.
constant_image	A constant image (one symbol class) is reproduced exactly.
corrupt_file_rejected	A damaged container raises an error instead of an image.
file_workflow_colour	File-based round trip starting from a color PNG.

F. Reproducibility

All results in Section VII can be regenerated from the repository. The evaluate command reproduces the quality sweep and the rate–distortion and comparison figures for any input image; scripts/generate_sample_image.py recreates the synthetic test image deterministically, and scripts/make_diagrams.py redraws the architecture, class and flow diagrams. The tests are run with `python -m unittest discover -s tests`. The software versions used are listed in Section VII-A; since the pipeline uses only deterministic operations (no random numbers), repeated runs on the same machine produce bit-identical compressed files.

G. Key Design Decisions

- **Quantization is included.** Huffman coding of unquantized, real-valued DCT output gives no saving; quantization is the only lossy step and quality 100 makes it nearly lossless.

- **Symbols are integers, not (run, size) pairs.** This is simpler and easy to explain; the EOB marker captures most of the zero-run benefit. Run-length symbols are listed as future work.
- **The frequency table is stored in the file** so that the decoder rebuilds an identical tree.
- **struct instead of pickle** for the container: safe, compact and language-neutral.
- **Vectorized block handling** with NumPy; the Huffman tree and bit-stream logic are in pure Python for clarity.
- **Grayscale only**, as specified.

VII. EXPERIMENTAL RESULTS

A. Setup and Test Images

All experiments were run with the code described above on a single development machine under Python 3.12 with NumPy 2.4, SciPy 1.17 and OpenCV 4.13. Three test images were used: a synthetic 512×512 picture (smooth gradients, a disc, a rectangle, a sharp line, a noisy texture patch and text) produced by scripts/generate_sample_image.py; a 512×600 portrait photograph (Photo 1, Matplotlib sample data); and a 640×427 outdoor photograph (Photo 2, scikit-learn sample data). The synthetic image is much smoother than a typical photograph and therefore compresses unrealistically well, so the photographs should be taken as more representative.

B. Evaluation Methodology

Each image was compressed at quality factors 10, 25, 50, 75 and 90. The recorded metrics were the compressed file size (including the 26-byte header and the frequency table), the compression ratio relative to the raw 8-bit samples ($H \cdot W$ bytes), the bit rate in bpp, and MSE and PSNR between the original grayscale image and the reconstruction. Ablation experiments removed individual design elements (zig-zag with end-of-block markers, differential DC coding, the transform itself) to measure their contribution. The files were then compared with those of the OpenCV JPEG encoder at the same nominal quality, and running times were measured for increasing image sizes. The entropy and bit-allocation analyses of Sections VII-E and VII-F were computed from the symbol streams of the sample image.

C. Compression Ratio and Quality

Table XI lists the results. As expected, higher quality lowers distortion and raises the file size (Fig. 4). At quality 50 the synthetic image reaches 26.6:1 at 38.8 dB, whereas the photographs reach only 10.8:1 (33.7 dB) and 6.9:1 (28.2 dB), because natural images contain more high-frequency detail and therefore more non-zero coefficients. The very steep rise in PSNR of Photo 1 between quality 50 and 90 may be related to the fact that this image was itself stored as a JPEG, so that its coefficients lie close to a quantization grid; this was not investigated further. Fig. 5 shows the same data as compression ratio versus quality factor.

TABLE XI
RATE–DISTORTION RESULTS (RAW SIZE: SYNTHETIC 262,144 B; PHOTO 1 307,200 B; PHOTO 2 273,280 B)

Image	*q*	Bytes	Ratio	bpp	MSE	PSNR (dB)
Synthetic	10	3,382	77.51:1	0.10	27.95	33.67
	25	6,407	40.92:1	0.20	13.49	36.83
	50	9,859	26.59:1	0.30	8.63	38.77
	75	17,656	14.85:1	0.54	5.40	40.81
	90	50,085	5.23:1	1.53	2.91	43.49
Photo 1	10	9,426	32.59:1	0.25	90.50	28.56
	25	17,598	17.46:1	0.46	46.58	31.45
	50	28,538	10.76:1	0.74	27.49	33.74
	75	56,977	5.39:1	1.48	3.44	42.77
	90	67,486	4.55:1	1.76	0.33	52.91
Photo 2	10	11,501	23.76:1	0.34	292.27	23.47
	25	23,678	11.54:1	0.69	174.03	25.72
	50	39,429	6.93:1	1.15	97.72	28.23
	75	59,677	4.58:1	1.75	40.56	32.05
	90	93,731	2.92:1	2.74	8.40	38.89

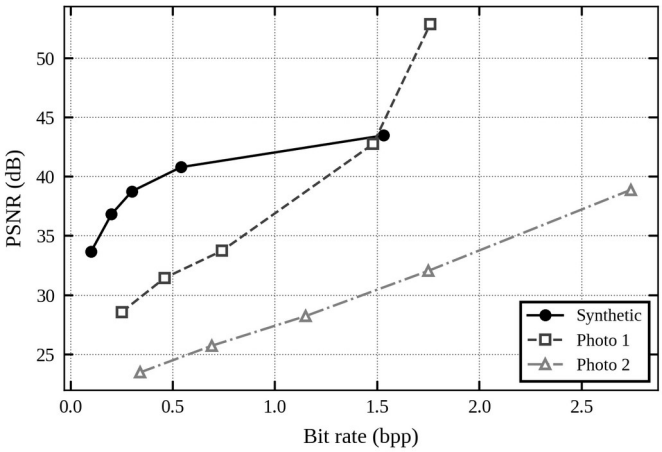


Fig. 4. Rate–distortion curves for the three test images (PSNR versus bits per pixel).

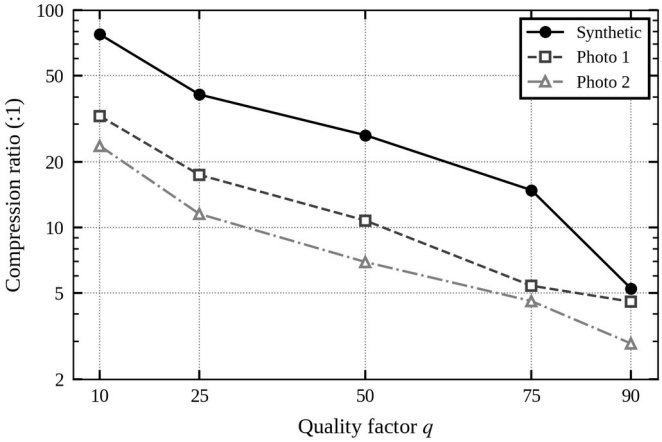


Fig. 5. Compression ratio versus quality factor q for the three test images (logarithmic vertical axis).

D. Ablation: Contribution of Each Design Element

To quantify the contribution of each stage, the Huffman payload size at quality 50 was measured for four symbol models (Table XII). Because Huffman coding is applied to individual values, the order in which values are emitted does not change the histogram, so raster order and zig-zag order give

identical sizes unless an end-of-block marker is used. Three conclusions follow:

TABLE XII
HUFFMAN PAYLOAD SIZE (BYTES) AT Q = 50 FOR DIFFERENT SYMBOL MODELS

Symbol model	Photo 1	Photo 2	Synthetic
Raw pixels, no transform (whole file)	273,157	265,971	234,189
All 64 quantized values per block	56,354	60,922	39,482
Zig-zag + EOB, raw DC	29,718	41,949	11,415
Zig-zag + EOB + DC difference (final)	27,096	38,435	8,977

- Transform and quantization reduce the size from roughly 234,000–273,000 bytes (1.03:1 to 1.12:1, essentially no compression) to 39,000–61,000 bytes.
- Zig-zag scanning with end-of-block markers removes a further 31% to 71% by eliminating the long runs of trailing zeros.
- Differential DC coding gives an additional 8% to 21%, because neighboring blocks have similar mean intensity.

The transform is therefore the indispensable stage: Huffman coding of raw pixels alone is nearly useless because the pixel histogram of a natural image is close to flat, whereas the quantized coefficient histogram is sharply peaked at zero. Fig. 6 visualizes Table XII: the transform removes about an order of magnitude of data, and zig-zag with EOB and DC differencing remove a further 31% to 71% and 8% to 21%, respectively.

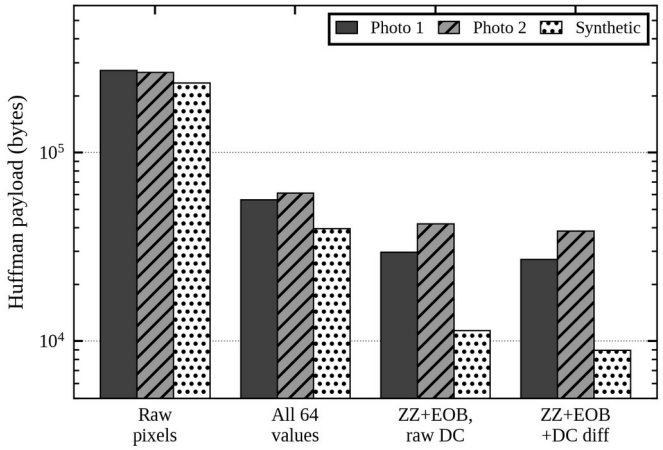


Fig. 6. Huffman payload size at q = 50 for the four symbol models of Table XII (logarithmic vertical axis; ZZ = zig-zag scan).

E. Entropy Efficiency of the Huffman Stage

Table XIII compares the average Huffman code length L with the empirical zero-order entropy H of the symbol stream for the sample image, and Table XIV gives further statistics of the same streams. The Huffman code is within 0.02–0.06 bit/symbol of the entropy (97.3–99.2% efficiency, $\eta = H/L$), consistent with the bound $H \le L < H + 1$. The redundancy is small because no single symbol dominates the distribution enough to force a large integer-rounding loss, although the EOB symbol (one per block) is frequent and receives a code of only 2–5 bits. Consequently the remaining inefficiency of the compressor lies in the *symbol model* (what is coded), not in the Huffman *code* (how it is coded): a better code for the same symbols could save at most a few percent, while a better symbol model, such as run-length pairs, can shorten the stream itself.

TABLE XIII
ENTROPY VERSUS AVERAGE HUFFMAN CODE LENGTH (SAMPLE IMAGE)

q	Symbols N	Distinct n	H (bit)	L (bit)	$L - H$	$\eta = H/L$
10	11,205	27	2.180	2.242	0.062	97.3%
25	18,017	60	2.567	2.620	0.053	98.0%
50	25,068	107	2.823	2.865	0.041	98.6%
75	45,633	194	2.795	2.819	0.024	99.2%
90	163,979	407	2.224	2.283	0.059	97.4%

TABLE XIV
SYMBOL STATISTICS AND TABLE OVERHEAD (SAMPLE IMAGE)

q	Zero coeffs.	Payload (B)	Table (B)	Table share	File (B)
10	98.1%	3,140	216	6.4%	3,382
25	96.7%	5,901	480	7.5%	6,407
50	95.2%	8,977	856	8.7%	9,859
75	91.9%	16,078	1,552	8.8%	17,656
90	76.0%	46,803	3,256	6.5%	50,085

At quality 10, 98.1% of all quantized coefficients are zero, which the zig-zag scan and the end-of-block marker reduce to a short symbol stream (11,205 symbols for 4,096 blocks). The frequency table is a small but non-negligible overhead of 6.4–8.8% of the file; storing canonical code lengths instead of frequencies would reduce it. Fig. 7 shows that L tracks H closely at every quality setting, with efficiency between 97.3% and 99.2%.

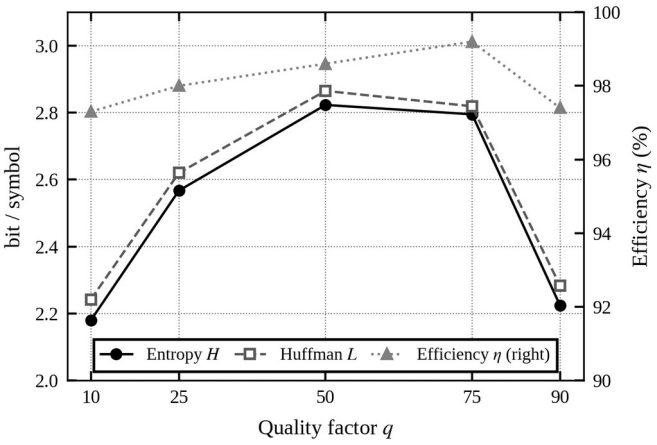


Fig. 7. Source entropy H, average Huffman code length L and coding efficiency η = H/L versus quality factor (sample image).

F. Bit Allocation and Interior Zeros

Table XV splits the payload bits among the three symbol classes. At low quality a large share goes to the EOB markers and DC differences, because each block still needs them; at high quality the AC symbols dominate (87% at quality 90). The last column shows the fraction of emitted AC symbols that are *interior zeros*, zeros that precede the last non-zero coefficient of a block and therefore must be coded one by one. It rises from 45% at quality 10 to 62% at quality 90.

TABLE XV
SHARE OF PAYLOAD BITS BY SYMBOL CLASS AND INTERIOR-ZERO FRACTION (SAMPLE IMAGE)

q	DC diff.	AC	EOB	EOB code length	Interior zeros / AC symbols
10	29.8%	37.5%	32.6%	2 bit	45.4%
25	20.8%	61.8%	17.4%	2 bit	49.6%
50	17.3%	65.6%	17.1%	3 bit	48.3%
75	13.3%	74.0%	12.7%	4 bit	53.6%
90	7.7%	86.9%	5.5%	5 bit	62.3%

This is the mechanism behind the gap to JPEG reported in Section VII-G. Baseline JPEG codes each non-zero coefficient together with the length of the preceding zero run as one symbol, so interior zeros cost nothing individually; in this implementation each of them is a separate symbol with its own code word. A run-length symbol model would remove the roughly half of all AC symbols that are interior zeros, at the price of a larger alphabet. Fig. 8 plots this split: the AC share grows from 37.5% to 86.9% as the quality rises.

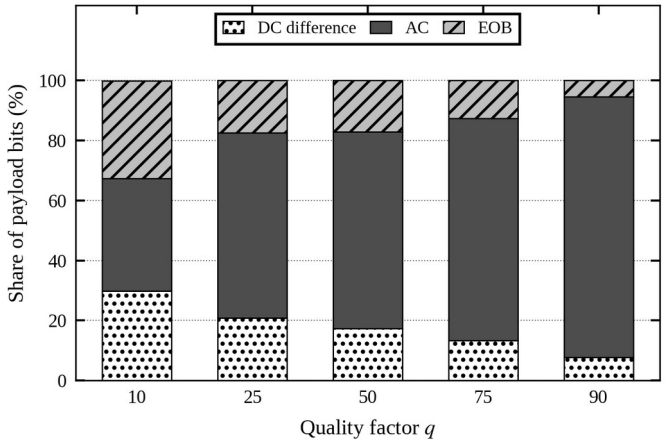


Fig. 8. Share of payload bits spent on DC differences, AC symbols and EOB markers versus quality factor (sample image).

G. Comparison with a Reference JPEG Encoder

The same grayscale images were encoded with OpenCV's JPEG encoder (libjpeg) at the same quality factors. Because this tool uses the same DCT, level shift and quantization tables, the reconstruction PSNR values agreed to within 0.01 dB in all tested cases, and the comparison isolates the effect of the entropy-coding and symbol-modeling stages. Table XVI compares file sizes against JPEG with optimized Huffman tables (IMWRITE_JPEG_OPTIMIZE), which is the fairest comparison because this tool also builds per-image tables.

TABLE XVI
FILE SIZE RELATIVE TO OPTIMIZED JPEG (THIS TOOL ÷ JPEG; > 1 MEANS LARGER)

Quality	Photo 1	Photo 2	Synthetic
10	1.12	1.08	1.25
50	1.11	1.07	1.15
90	1.04	1.06	1.15

The files produced here are 4% to 25% larger than optimized JPEG. Against libjpeg's default (non-optimized) tables the ratio ranged from 0.72 to 1.08, so this tool was sometimes smaller. The remaining gap to optimized JPEG is attributable to the absence of joint (run, size) symbols (Section VII-F), to the explicit frequency table, and to the stored magnitude of every coefficient, whereas JPEG codes a magnitude category plus raw bits. Since the transform and quantization are identical, these results indicate that the entropy-coding stage of the project is reasonable but not state of the art. The ratios of Table XVI are plotted in Fig. 9; the gap narrows as the quality factor increases for the photographs.

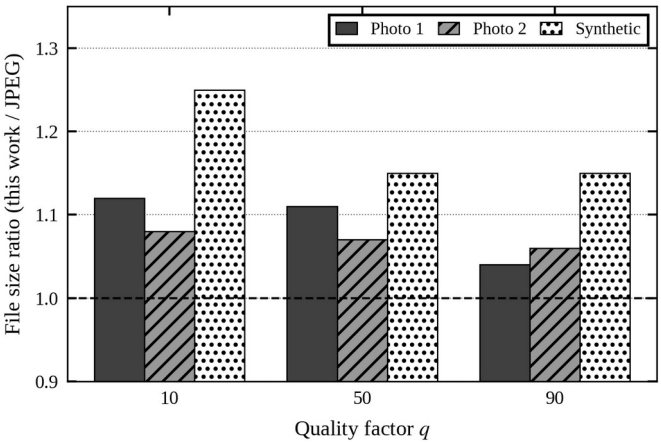


Fig. 9. File size of this tool relative to libjpeg with optimized Huffman tables at identical quantization (values above 1 are larger).

H. Running Time

TABLE XVII
RUNNING TIME VERSUS IMAGE SIZE (Q = 50; SYNTHETIC IMAGE TILED)

Image size	Blocks	Compress (s)	Decompress (s)
256×256	1,024	0.008	0.005
512×512	4,096	0.022	0.022
1024×1024	16,384	0.085	0.085
2048×2048	65,536	0.373	0.282

Each doubling of the side length multiplies the time by about four (3.9× from 512 to 1024 and 4.4× from 1024 to 2048), i.e. the running time is linear in the number of pixels, as predicted in Section VIII. The absolute values are specific to the test machine; the tiled images are periodic, so only the timing, not the file size, of these larger images should be interpreted. Fig. 10 confirms the linear trend on log–log axes: the measured curves run parallel to the linear reference.

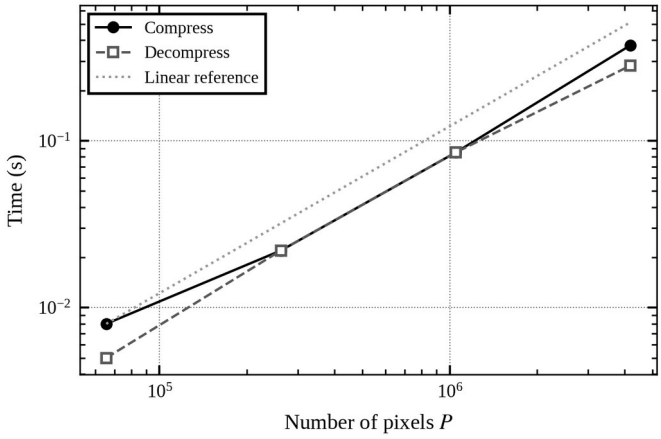


Fig. 10. Compression and decompression time versus number of pixels (log–log axes) with a linear-scaling reference line.

I. Visual Quality

Fig. 11 shows the original image and the reconstructions at quality 10, 50 and 90 for Photo 1 and for the synthetic image. Quality 90 is visually indistinguishable from the original and quality 50 shows only minor softening. At quality 10 the characteristic 8×8 blocking artifacts are visible in smooth regions, a known limitation of block transforms at low bit rates that follows directly from coding the blocks independently (Section III-F). Text and sharp edges in the synthetic image show ringing at low quality, because the quantization of high-

frequency coefficients removes the harmonics needed to represent a step edge.



Fig. 11. Photo 1: original and reconstructions at quality 10, 50 and 90 (compression ratio and PSNR above each image).

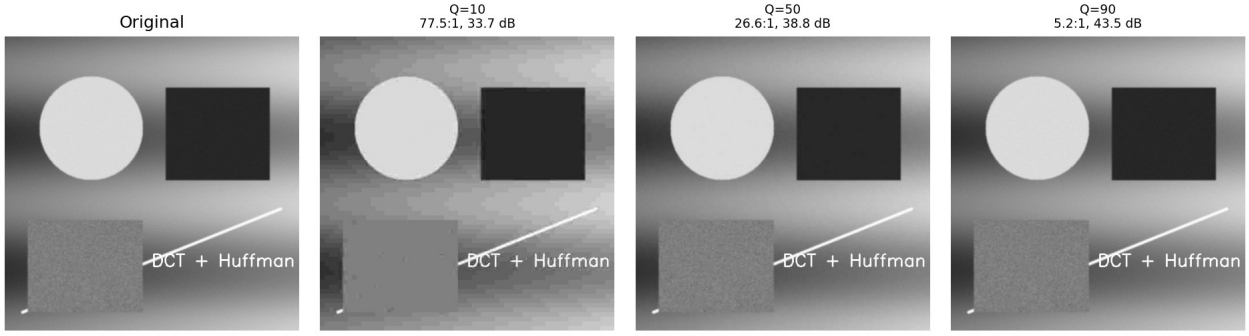


Fig. 12. Synthetic image: original and reconstructions at quality 10, 50 and 90.

J. Energy Compaction

Why the pipeline works is visible in the distribution of transform energy. For the sample image, the DC coefficient alone carries 94.4% of the total energy of the level-shifted blocks, the first three coefficients in zig-zag order 97.5%, the first ten 99.2% and the first 28 99.7%; the remaining 36 high-frequency coefficients together carry less than 0.3%. This concentration is what the zig-zag scan and the EOB marker exploit. Because the sample image is smooth by construction, the compaction is stronger than for natural photographs, which is consistent with their lower compression ratios in Table XI. Fig. 13 plots the cumulative energy against the number of retained coefficients.

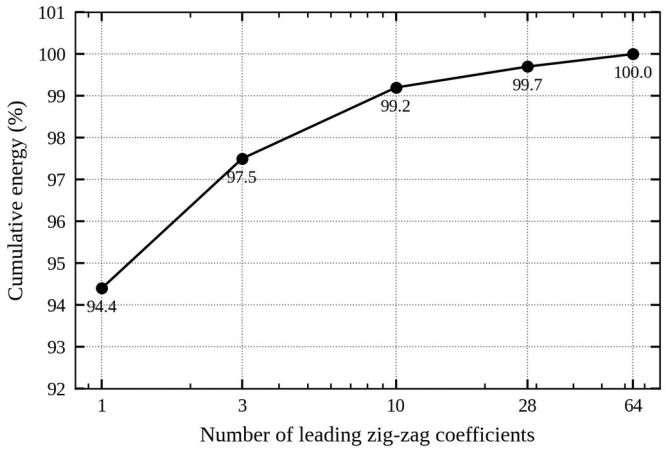


Fig. 13. Cumulative share of transform energy captured by the leading zig-zag coefficients (sample image).

VIII. COMPLEXITY ANALYSIS

Let n be the number of distinct symbols, N the length of the symbol stream, P the number of pixels, $B \approx P/64$ the number of blocks and $b = 8$ the block size. Table XVIII gives the required summary; the interpretation of the variables in each row is stated below the table.

TABLE XVIII
COMPLEXITY OF THE MAIN COMPONENTS

Component	Time	Space
Huffman tree build	$O(n \log n)$	$O(n)$
Encoding	$O(n)$	$O(n)$
Decoding	$O(n)$	$O(n)$
Block processing	$O(b)$	$O(b)$
DCT transform	$O(b^2)$	$O(b^2)$

In the rows for encoding and decoding, "n" denotes the length of the data being processed (N above); in "block processing", "b" is the number of blocks (B); and in "DCT transform", b^2 is the number of samples in one block. For this implementation the bounds are, more precisely:

- **Tree build:** n insertions and $n - 1$ merges, each $O(\log n)$, give $O(n \log n)$ time and $O(n)$ space. Code generation is $O(n)$ in the number of nodes and $O(nL)$ space for the code strings, where $L \le n - 1$ is the maximum code length.
- **Encoding:** $O(N)$ symbol look-ups plus $O(N \cdot \bar{L})$ bit-string work, where $\bar{L} \approx 2$–3 bit is the average code length, i.e. $O(N)$ time and space.
- **Decoding:** one tree step per bit, $O(N \cdot \bar{L}) = O(N)$ time and $O(N)$ space.
- **Block processing:** splitting and merging are single passes, $O(P) = O(B)$ for fixed b.
- **DCT per block:** a separable DCT written as two matrix products costs $O(b^3)$ per block; a fast algorithm costs $O(b^2 \log b)$ [7]. Since b is the constant 8, the cost per block is $O(1)$ and the cost for the whole image is $O(P)$. The blocks are independent, so the work parallelizes trivially.

- **Whole pipeline:** O($P + n \log n$) time and O(P) space for compression, O(P) for decompression. Because n is at most a few hundred for realistic quality settings (407 at quality 90 on the sample image; at most about 4,100 for 8-bit input) and P is in the hundreds of thousands or millions, the pipeline is effectively linear in image size, which matches the measurements of Section VII-H.

TABLE XIX
DETAILED STAGE-BY-STAGE COMPLEXITY

Stage	Time	Space / note
Load, grayscale, pad	O(P)	O(P)
Divide / merge blocks	O(P)	O(P); reshape copy
DCT of whole image	O($P \log b$) = O(P)	O(P)
Quantize / dequantize	O(P)	O(P)
Symbol generation	O(P)	O(N), $N \le P + B$
Frequency count	O(N)	O(n); hash table
Huffman build	O($n \log n$)	O(n)
Encode / decode	O(N)	O(N)

The frequency table has a space cost of 8 bytes per distinct symbol (Section VII-E), which is a constant-factor overhead on the compressed file but does not affect the asymptotic bounds.

IX. DISCUSSION

A. Advantages

- **Transparent and modular:** each stage is a separate, testable class, which makes the system suitable for teaching and experimentation.
- **Good compression for its simplicity:** ratios of 7:1 to 11:1 on natural photographs at PSNR above 28 dB, within roughly 4–25% of optimized JPEG at essentially identical distortion.
- **Tunable quality:** one integer parameter spans near-lossless to heavily compressed output.
- **Optimal prefix code per image:** the Huffman table adapts to the statistics of each image and operates within 0.06 bit/symbol of the entropy.
- **Linear running time and a trivially parallel block transform.**
- **Robust and safe:** a self-describing format, input validation, explicit error messages and no unsafe deserialization.
- **Reproducible:** tests, scripts and documentation are included.

B. Limitations

- **Lossy:** quantization discards information; exact recovery is not possible except approximately at quality 100.
- **Grayscale only:** color images are converted to luminance and the chrominance information is lost.
- **Blocking artifacts** appear at low quality because blocks are coded independently.
- **Symbol model:** the absence of run-length symbols and of context modeling leaves 4–25% on the table compared with optimized JPEG.
- **Integer code lengths:** Huffman coding cannot go below one bit per symbol and cannot reach the entropy exactly, which matters when a symbol is highly probable.
- **Table overhead:** storing frequencies costs 8 bytes per distinct symbol, 6–9% of the file in the measured cases.
- **Pure-Python entropy stage:** tree walking and bit-string construction are slower than a compiled implementation for very large images.
- **Limited evaluation:** three images and PSNR only.

C. Threats to Validity

- Only three images were used, one of them synthetic; results on larger, more diverse data sets may differ. The entropy, bit-allocation and energy analyses of Sections VII-E, VII-F and VII-J use the synthetic image only and quantify mechanisms rather than expected photographic performance.
- PSNR is an imperfect proxy for visual quality; SSIM or subjective tests were not performed.
- Timings come from a single machine and a single run each, without repetition statistics.
- The JPEG comparison covers grayscale only and one library implementation.

D. Choosing the Quality Factor

The measured rate–distortion points (Table XI) give practical guidance. For the smooth synthetic image, quality 25 already exceeds 36 dB at 0.20 bpp; for Photo 1, quality 75 reaches 42.8 dB at 1.48 bpp; and for the more detailed Photo 2, even quality 90 reaches only 38.9 dB at 2.74 bpp. A single quality number therefore does not correspond to a fixed fidelity: the required quality depends on image content, and an application that needs a guaranteed PSNR should search over the quality factor, which the evaluate command supports. In these measurements, between quality 10 and 50 each doubling of the file size buys roughly 2–3 dB of PSNR.

E. Design Alternatives Considered

Table XX summarizes alternatives to the choices made in this work. The effects listed are expectations derived from the analysis of Sections VII-E and VII-F and from the literature, not measurements; verifying them is part of the future work.

TABLE XX
DESIGN ALTERNATIVES AND EXPECTED EFFECT

Alternative	Expected effect	Cost	Status
Run-length (run, size) symbols	Removes interior-zero symbols; closes most of the gap to JPEG	Larger alphabet; more code	Future work
Canonical Huffman codes	Table stores code lengths instead of 4-byte frequencies; smaller file	Small decoder change	Future work
Arithmetic coding	Approaches entropy; gains most at low rates	Higher complexity	Future work
Static (fixed) Huffman tables	No table stored; loses image adaptivity	None	Not adopted
Larger blocks (16×16)	Better compaction on smooth areas; more ringing	Higher per-block cost	Not evaluated

X. FUTURE WORK

- **Color support:** convert to YCbCr, subsample the chrominance planes and code the three planes separately.
- **Better symbol model:** joint (zero-run, size) symbols as in baseline JPEG, which Section VII-F suggests would close much of the gap to optimized JPEG.
- **Canonical Huffman codes:** store code lengths instead of frequencies to shrink the table [4].
- **Adaptive or arithmetic coding:** approach the entropy more closely [5].
- **Deblocking filter and adaptive quantization** to improve visual quality at low rates.
- **Variable block size** (4×4 to 32×32), chosen per region.

- **Parallel processing** of blocks with multiprocessing or a compiled extension.
- **Additional metrics and data:** SSIM [13] and evaluation on standard data sets such as the Kodak suite.
- **Graphical interface** for interactive quality selection and side-by-side viewing.

XI. CONCLUSION

This paper presented a complete, tested and documented image compression tool that combines block-wise DCT processing with Huffman coding. The design follows the classical transform-coding architecture: independent 8×8 blocks (divide and conquer), an orthonormal DCT, quality-controlled quantization, differential DC coding with zig-zag scanning and end-of-block markers, and a greedy Huffman code built with a min-heap. The decompressor reverses each step and reproduces the image with controllable distortion.

The experiments confirmed the expected behavior. Huffman coding of raw pixels is nearly useless (1.03:1 to 1.12:1), whereas the transform, quantization and a structured symbol stream raise the ratio to 6.9:1 and 10.8:1 on two natural photographs at quality 50, with PSNR of 28.2 dB and 33.7 dB. The end-of-block mechanism and differential DC coding each contributed measurably. The Huffman code itself operates within 0.02–0.06 bit/symbol of the entropy, so the 4–25% size gap to optimized JPEG at identical PSNR is due to the symbol model, chiefly the roughly 45–62% of AC symbols that are interior zeros, rather than to the coding stage. The running time grew linearly with the number of pixels, in agreement with the analysis.

Beyond the numbers, the project demonstrates how a greedy algorithm and a divide-and-conquer decomposition combine in a real application, and the modular code base provides a solid foundation for the extensions listed in Section X.

APPENDIX A: USER GUIDE

Installation.

Python 3.11 or newer is required. In a virtual environment, install the four dependencies:

```
python -m venv venv
venv\Scripts\activate        (Windows)
source venv/bin/activate     (macOS / Linux)
pip install -r requirements.txt
```

Commands.

```
# compress (default quality 50)
python src/main.py compress input/sample_image.jpg
# choose quality and output path
python src/main.py compress input/sample_image.jpg
       -q 75 -o output/compressed.bin
# decompress; --original prints MSE and PSNR
python src/main.py decompress output/compressed.bin
       --original input/sample_image.jpg
# quality sweep, table and figures in output/
python src/main.py evaluate input/sample_image.jpg
# unit tests
python -m unittest discover -s tests
```

Sample session.

```
$ python src/main.py compress input/sample_image.jpg -q 50
Compressed input/sample_image.jpg -> output/compressed.bin
  Image size        : 512 x 512
  Quality           : 50
  Raw pixel bytes   : 262144
  Compressed bytes  : 9859
  Compression ratio : 26.59:1 (vs raw grayscale)
  Bits per pixel    : 0.301

$ python src/main.py decompress output/compressed.bin
       --original input/sample_image.jpg
Reconstructed image saved to output/reconstructed.png
  MSE  : 8.633
  PSNR : 38.77 dB
```

Input handling.

Any image format supported by OpenCV is accepted. Color, 16-bit and alpha-channel inputs are converted to 8-bit grayscale. Image sizes that are not a multiple of 8 are edge-padded on encoding and cropped on decoding. Quality 100 gives near-lossless output; quality 1 gives the smallest file.

APPENDIX B: SOURCE LISTINGS

The listings below are the core modules as executed in the experiments, with docstrings omitted for space; very long lines are wrapped. The complete files, with documentation, are in the project repository.

```
from typing import Optional

class HuffmanNode:

    __slots__ = ("frequency", "value", "left", "right")

    def __init__(self, frequency: int, value: Optional[int] =
        None,
                 left: Optional["HuffmanNode"] = None,
                 right: Optional["HuffmanNode"] = None) ->
                None:
        self.frequency = frequency
        self.value = value
        self.left = left
        self.right = right

    @property
    def is_leaf(self) -> bool:
                return self.left is None and self.right is
                None

    def __lt__(self, other: "HuffmanNode") -> bool:
        return self.frequency < other.frequency

    def __repr__(self) -> str:
        return f"HuffmanNode(freq={self.frequency},
            value={self.value})"
```

Listing 3. `huffman_node.py` — Huffman node (HuffmanNode).

```
import heapq
from typing import Dict, Iterable, List, Optional

from huffman_node import HuffmanNode

class HuffmanTree:

    def __init__(self) -> None:
        self.root: Optional[HuffmanNode] = None
        self.codes: Dict[int, str] = {}

    def build(self, frequencies: Dict[int, int]) ->
        "HuffmanTree":
                if not frequencies:
            raise ValueError("Cannot build a Huffman tree from
                no symbols.")
        heap = []
        for order, (symbol, freq) in
            enumerate(sorted(frequencies.items())):
            heapq.heappush(heap, (freq, order,
                HuffmanNode(freq, symbol)))
        counter = len(heap)
        while len(heap) > 1:
            f1, _, left = heapq.heappop(heap)
            f2, _, right = heapq.heappop(heap)
            parent = HuffmanNode(f1 + f2, None, left, right)
            heapq.heappush(heap, (parent.frequency, counter,
                parent))
            counter += 1
        self.root = heap[0][2]
        self.generate_codes()
        return self

    def generate_codes(self) -> Dict[int, str]:
                if self.root is None:
            raise ValueError("Tree has not been built yet.")
        self.codes = {}
```

```
        if self.root.is_leaf:  # degenerate case: only one
            distinct symbol
            self.codes[self.root.value] = "0"
            return self.codes
        stack = [(self.root, "")]
        while stack:
            node, prefix = stack.pop()
            if node.is_leaf:
                self.codes[node.value] = prefix
            else:
                stack.append((node.right, prefix + "1"))
                stack.append((node.left, prefix + "0"))
        return self.codes

    def decode(self, bits: Iterable[int]) -> List[int]:
                if self.root is None:
            raise ValueError("Tree has not been built yet.")
        root = self.root
        symbols: List[int] = []
        if root.is_leaf:
            return [root.value for _ in bits]
        node = root
        for bit in bits:
            node = node.right if bit else node.left
            if node.is_leaf:
                symbols.append(node.value)
                node = root
        if node is not root:
            raise ValueError("Bit stream ended inside a code
                word.")
        return symbols

    def average_code_length(self) -> float:
                if self.root is None or self.root.frequency ==
                0:
            return 0.0
        total = self.root.frequency
        stack = [(self.root, 0)]
        weighted = 0
        while stack:
            node, depth = stack.pop()
            if node.is_leaf:
                weighted += node.frequency * max(depth, 1)
            else:
                stack.append((node.left, depth + 1))
                stack.append((node.right, depth + 1))
        return weighted / total
```

Listing 4. `huffman_tree.py` — Huffman tree construction, code generation and decoding (HuffmanTree).

```
from collections import Counter
from dataclasses import dataclass
from typing import Dict, List

import numpy as np

from huffman_tree import HuffmanTree
from utils import EOB_SYMBOL, ZIGZAG

@dataclass
class EncodedData:

    payload: bytes
    n_bits: int
    frequencies: Dict[int, int]
    tree: HuffmanTree

class Encoder:

    def blocks_to_symbols(self, quantized: np.ndarray) ->
        List[int]:
                n_blocks = quantized.shape[0]
        zz = quantized.reshape(n_blocks, -1)[:, ZIGZAG]
        symbols: List[int] = []
        previous_dc = 0
        for row in zz:
            dc = int(row[0])
            symbols.append(dc - previous_dc)
            previous_dc = dc
            nonzero = np.flatnonzero(row[1:])
            if nonzero.size:
                symbols.extend(row[1:nonzero[-1] +
                2].tolist())
            symbols.append(EOB_SYMBOL)
        return symbols

    @staticmethod
    def calculate_frequencies(symbols: List[int]) -> Dict[int,
        int]:
                return dict(Counter(symbols))

    def encode(self, symbols: List[int]) -> EncodedData:
                if not symbols:
            raise ValueError("Nothing to encode.")
        frequencies = self.calculate_frequencies(symbols)
        tree = HuffmanTree().build(frequencies)
        codes = tree.codes
        bitstring = "".join(codes[s] for s in symbols)
        bits = np.frombuffer(bitstring.encode("ascii"),
            dtype=np.uint8) - 48
        payload = np.packbits(bits).tobytes()
        return EncodedData(payload, len(bitstring),
            frequencies, tree)
```

Listing 5. `encoder.py` — Symbol stream, frequency count and bit packing (Encoder).

```
from typing import Dict, List

import numpy as np

from huffman_tree import HuffmanTree
from utils import EOB_SYMBOL, INVERSE_ZIGZAG

class Decoder:

    def decode(self, payload: bytes, n_bits: int,
               frequencies: Dict[int, int]) -> List[int]:
                tree = HuffmanTree().build(frequencies)
        bits = np.unpackbits(np.frombuffer(payload,
            dtype=np.uint8))[:n_bits]
        return tree.decode(bits.tolist())

    def symbols_to_blocks(self, symbols: List[int], n_blocks:
        int) -> np.ndarray:
                blocks = np.zeros((n_blocks, 64),
                dtype=np.int32)
        pos, total, previous_dc = 0, len(symbols), 0
        for i in range(n_blocks):
            if pos >= total:
                raise ValueError("Corrupt data: ran out of
                symbols.")
            previous_dc += symbols[pos]
            pos += 1
            zz = np.zeros(64, dtype=np.int32)
            zz[0] = previous_dc
            k = 1
            while pos < total and symbols[pos] != EOB_SYMBOL:
                if k >= 64:
                    raise ValueError("Corrupt data: block has
                too many values.")
                zz[k] = symbols[pos]
                k += 1
                pos += 1
            if pos >= total:
                raise ValueError("Corrupt data: missing
                end-of-block marker.")
            pos += 1  # skip EOB
            blocks[i] = zz[INVERSE_ZIGZAG]
        if pos != total:
            raise ValueError("Corrupt data: trailing symbols
                after last block.")
        return blocks.reshape(n_blocks, 8, 8)
```

Listing 6. `decoder.py` — Tree rebuild, bit decoding and block reconstruction (Decoder).

```
import numpy as np
from scipy.fft import dctn, idctn

#: Standard JPEG luminance quantisation table (quality 50).
BASE_QUANT_TABLE = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99],
], dtype=np.float64)

class TransformBlock:

    def __init__(self, quality: int = 50) -> None:
        if not 1 <= int(quality) <= 100:
            raise ValueError("Quality must be an integer in
```

```
                [1, 100].")
        self.quality = int(quality)
        self.quant_table = self._scaled_table(self.quality)

    @staticmethod
    def _scaled_table(quality: int) -> np.ndarray:
        scale = 5000 / quality if quality < 50 else 200 - 2 *
            quality
        table = np.floor((BASE_QUANT_TABLE * scale + 50) /
            100)
        return np.clip(table, 1, 255)

    def apply_dct(self, blocks: np.ndarray) -> np.ndarray:
                shifted = blocks.astype(np.float64) - 128.0
        return dctn(shifted, type=2, norm="ortho", axes=(-2,
            -1))

    def apply_inverse_dct(self, coeffs: np.ndarray) ->
        np.ndarray:
                pixels = idctn(coeffs, type=2, norm="ortho",
                axes=(-2, -1)) + 128.0
        return np.clip(np.rint(pixels), 0,
            255).astype(np.uint8)

    def quantize(self, coeffs: np.ndarray) -> np.ndarray:
                return np.rint(coeffs /
                self.quant_table).astype(np.int32)

    def dequantize(self, quantized: np.ndarray) -> np.ndarray:
                return quantized.astype(np.float64) *
                self.quant_table

    def forward(self, blocks: np.ndarray) -> np.ndarray:
                return self.quantize(self.apply_dct(blocks))

    def inverse(self, quantized: np.ndarray) -> np.ndarray:
                return self.apply_inverse_dct(self.dequantize(
                quantized))
```

Listing 7. `transform_block.py` — DCT and quantization (TransformBlock).

```
from pathlib import Path
from typing import Tuple

import cv2
import numpy as np

from utils import BLOCK_SIZE

class ImageProcessor:

    def __init__(self, block_size: int = BLOCK_SIZE) -> None:
        self.block_size = block_size

    def load_image(self, path) -> np.ndarray:
                path = Path(path)
        if not path.is_file():
            raise FileNotFoundError(f"Input image not found:
                {path}")
        image = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
        if image is None:
            raise ValueError(f"Could not decode image:
                {path}")
        return image

    def to_grayscale(self, image: np.ndarray) -> np.ndarray:
                if image.ndim == 3:
            if image.shape[2] == 4:
                image = cv2.cvtColor(image,
                cv2.COLOR_BGRA2GRAY)
            else:
                image = cv2.cvtColor(image,
                cv2.COLOR_BGR2GRAY)
        if image.dtype != np.uint8:
            peak = 65535.0 if image.dtype == np.uint16 else
                max(float(image.max()), 1.0)
            image = np.clip(image.astype(np.float64) / peak *
                255.0, 0, 255).astype(np.uint8)
        return image

    def load_grayscale(self, path) -> np.ndarray:
                return self.to_grayscale(self.load_image(path)
                )

    def padded_shape(self, shape: Tuple[int, int]) ->
        Tuple[int, int]:
                b = self.block_size
        return (-(-shape[0] // b) * b, -(-shape[1] // b) * b)

    def divide_into_blocks(self, gray: np.ndarray) ->
        np.ndarray:
                b = self.block_size
        ph, pw = self.padded_shape(gray.shape)
        padded = np.pad(gray, ((0, ph - gray.shape[0]), (0, pw
            - gray.shape[1])),
                        mode="edge")
        blocks = padded.reshape(ph // b, b, pw // b,
            b).swapaxes(1, 2)
        return blocks.reshape(-1, b, b)

    def reconstruct(self, blocks: np.ndarray, shape:
        Tuple[int, int]) -> np.ndarray:
                b = self.block_size
        ph, pw = self.padded_shape(shape)
        rows, cols = ph // b, pw // b
        if blocks.shape[0] != rows * cols:
            raise ValueError("Block count does not match the
                image size.")
        merged = blocks.reshape(rows, cols, b, b).swapaxes(1,
            2).reshape(ph, pw)
        return merged[:shape[0], :shape[1]]

    @staticmethod
    def save_image(path, image: np.ndarray) -> None:
                path = Path(path)
        path.parent.mkdir(parents=True, exist_ok=True)
        if not cv2.imwrite(str(path), image):
            raise IOError(f"Could not write image: {path}")
```

Listing 8. `image_processor.py` — Image loading, padding and block partitioning (ImageProcessor).

```
from pathlib import Path
from typing import Dict, Optional

import numpy as np

from encoder import Encoder
from image_processor import ImageProcessor
from transform_block import TransformBlock
from utils import pack_container

class CompressionManager:

    def __init__(self, quality: int = 50) -> None:
        self.quality = quality
        self.processor = ImageProcessor()
        self.transform = TransformBlock(quality)
        self.encoder = Encoder()

    def compress_array(self, gray: np.ndarray) -> bytes:
                blocks =
                self.processor.divide_into_blocks(gray)
        quantized = self.transform.forward(blocks)
        symbols = self.encoder.blocks_to_symbols(quantized)
        encoded = self.encoder.encode(symbols)
        return pack_container(gray.shape[0], gray.shape[1],
            self.quality,
                              encoded.frequencies,
                encoded.payload,
                              encoded.n_bits)

    def compress(self, input_path, output_path: Optional[str]
        = None) -> Dict:
                input_path = Path(input_path)
        output_path = Path(output_path or
            "output/compressed.bin")
        gray = self.processor.load_grayscale(input_path)
        data = self.compress_array(gray)

        output_path.parent.mkdir(parents=True, exist_ok=True)
        output_path.write_bytes(data)

        raw_bytes = gray.size
        return {
            "input": str(input_path),
            "output": str(output_path),
            "shape": gray.shape,
            "quality": self.quality,
            "raw_bytes": raw_bytes,
            "input_file_bytes": input_path.stat().st_size,
            "compressed_bytes": len(data),
            "ratio_vs_raw": raw_bytes / len(data),
            "bits_per_pixel": 8.0 * len(data) / raw_bytes,
        }
```

Listing 9. `compression_manager.py` — Compression pipeline (CompressionManager).

```
from pathlib import Path
from typing import Optional

import numpy as np

from decoder import Decoder
from image_processor import ImageProcessor
from transform_block import TransformBlock
from utils import unpack_container

class DecompressionManager:

    def __init__(self) -> None:
        self.processor = ImageProcessor()
        self.decoder = Decoder()

    def decompress_bytes(self, data: bytes) -> np.ndarray:
                info = unpack_container(data)
        shape = (info["height"], info["width"])
        n_blocks = int(np.prod(np.array(self.processor.padded_
            shape(shape))
                               // self.processor.block_size))
        symbols = self.decoder.decode(info["payload"],
            info["n_bits"],
                                      info["frequencies"])
        quantized = self.decoder.symbols_to_blocks(symbols,
            n_blocks)
        pixels = TransformBlock(info["quality"]).inverse(quant
            ized)
        return self.processor.reconstruct(pixels, shape)

    def decompress(self, input_path, output_path:
        Optional[str] = None) -> np.ndarray:
                input_path = Path(input_path)
        if not input_path.is_file():
            raise FileNotFoundError(f"Compressed file not
                found: {input_path}")
        image = self.decompress_bytes(input_path.read_bytes())
        self.processor.save_image(output_path or
            "output/reconstructed.png", image)
        return image
```

Listing 10. `decompression_manager.py` — Decompression pipeline (DecompressionManager).

```
import struct
from typing import Dict

import numpy as np

BLOCK_SIZE = 8
#: Special "end of block" symbol. It lies outside the range of
    any
#: quantised DCT coefficient of an 8-bit image, so it cannot
    collide.
EOB_SYMBOL = 32768

MAGIC = b"HIMG"
VERSION = 1
_HEADER = struct.Struct("<4sBIIBIQ")
_SYMBOL = struct.Struct("<iI")

def zigzag_order(n: int = BLOCK_SIZE) -> np.ndarray:
        coords = [(r, c) for r in range(n) for c in range(n)]
    coords.sort(key=lambda rc: (rc[0] + rc[1],
                                rc[0] if (rc[0] + rc[1]) % 2
                else -rc[0]))
    return np.array([r * n + c for r, c in coords],
        dtype=np.int64)

ZIGZAG = zigzag_order()
INVERSE_ZIGZAG = np.argsort(ZIGZAG)

def mse(a: np.ndarray, b: np.ndarray) -> float:
        diff = a.astype(np.float64) - b.astype(np.float64)
    return float(np.mean(diff ** 2))

def psnr(a: np.ndarray, b: np.ndarray) -> float:
        error = mse(a, b)
    if error == 0:
        return float("inf")
    return float(10.0 * np.log10(255.0 ** 2 / error))

def pack_container(height: int, width: int, quality: int,
                   frequencies: Dict[int, int], payload:
                bytes,
                   n_bits: int) -> bytes:
        parts = [_HEADER.pack(MAGIC, VERSION, height, width,
            quality,
                              len(frequencies), n_bits)]
    for symbol, freq in sorted(frequencies.items()):
        parts.append(_SYMBOL.pack(symbol, freq))
    parts.append(payload)
    return b"".join(parts)

def unpack_container(data: bytes) -> dict:
        if len(data) < _HEADER.size:
        raise ValueError("File too small to be a valid
            compressed image.")
    magic, version, height, width, quality, n_symbols, n_bits
        = \
        _HEADER.unpack_from(data, 0)
    if magic != MAGIC:
        raise ValueError("Bad magic number: not a compressed
            image file.")
    if version != VERSION:
        raise ValueError(f"Unsupported container version
            {version}.")
    offset = _HEADER.size
    table_end = offset + n_symbols * _SYMBOL.size
    if table_end > len(data):
        raise ValueError("Corrupt file: frequency table is
            truncated.")
    frequencies = {}
    for i in range(n_symbols):
        symbol, freq = _SYMBOL.unpack_from(data, offset + i *
            _SYMBOL.size)
        frequencies[symbol] = freq
    payload = data[table_end:]
    if len(payload) * 8 < n_bits:
        raise ValueError("Corrupt file: payload is shorter
            than declared.")
    return {"height": height, "width": width, "quality":
        quality,
            "frequencies": frequencies, "payload": payload,
                "n_bits": n_bits}
```

Listing 11. `utils.py` — Zig-zag order, metrics and container format.

APPENDIX C: REFERENCE TABLES

Table XXI lists the baseline JPEG luminance quantization table used by TransformBlock (quality 50) and Table XXII gives the zig-zag scan index of each coefficient position, i.e. the order in which the coefficients are emitted into the symbol stream.

TABLE XXI
BASELINE QUANTIZATION TABLE Q_50 (ROWS: V, COLUMNS: U)

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0	16	11	10	16	24	40	51	61
1	12	12	14	19	26	58	60	55
2	14	13	16	24	40	57	69	56
3	14	17	22	29	51	87	80	62
4	18	22	37	56	68	109	103	77
5	24	35	55	64	81	104	113	92
6	49	64	78	87	103	121	120	101
7	72	92	95	98	112	100	103	99

TABLE XXII
ZIG-ZAG SCAN POSITION OF EACH COEFFICIENT

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0	0	1	5	6	14	15	27	28
1	2	4	7	13	16	26	29	42
2	3	8	12	17	25	30	41	43
3	9	11	18	24	31	40	44	53
4	10	19	23	32	39	45	52	54
5	20	22	33	38	46	51	55	60
6	21	34	37	47	50	56	59	61
7	35	36	48	49	57	58	62	63

Position 0 is the DC coefficient. The scan moves right from (0, 0), then diagonally down-left, alternating direction on each anti-diagonal, so that coefficients of similar spatial frequency

are adjacent and the high-frequency positions, which are almost always zero, form the tail of the sequence.

REFERENCES

[1] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948; and no. 4, pp. 623–656, Oct. 1948.

[2] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.

[3] R. G. Gallager, “Variations on a theme by Huffman,” *IEEE Trans. Inf. Theory*, vol. IT-24, no. 6, pp. 668–674, Nov. 1978.

[4] E. S. Schwartz and B. Kallick, “Generating a canonical prefix encoding,” *Commun. ACM*, vol. 7, no. 3, pp. 166–169, Mar. 1964.

[5] I. H. Witten, R. M. Neal, and J. G. Cleary, “Arithmetic coding for data compression,” *Commun. ACM*, vol. 30, no. 6, pp. 520–540, Jun. 1987.

[6] N. Ahmed, T. Natarajan, and K. R. Rao, “Discrete cosine transform,” *IEEE Trans. Comput.*, vol. C-23, no. 1, pp. 90–93, Jan. 1974.

[7] C. Loeffler, A. Ligtenberg, and G. S. Moschytz, “Practical fast 1-D DCT algorithms with 11 multiplications,” in *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, Glasgow, U.K., 1989, pp. 988–991.

[8] K. R. Rao and P. Yip, *Discrete Cosine Transform: Algorithms, Advantages, Applications*. San Diego, CA, USA: Academic, 1990.

[9] G. K. Wallace, “The JPEG still picture compression standard,” *IEEE Trans. Consum. Electron.*, vol. 38, no. 1, pp. xviii–xxxiv, Feb. 1992.

[10] W. B. Pennebaker and J. L. Mitchell, *JPEG: Still Image Data Compression Standard*. New York, NY, USA: Van Nostrand Reinhold, 1993.

[11] A. Skodras, C. Christopoulos, and T. Ebrahimi, “The JPEG 2000 still image compression standard,” *IEEE Signal Process. Mag.*, vol. 18, no. 5, pp. 36–58, Sep. 2001.

[12] D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Boston, MA, USA: Kluwer, 2002.

[13] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[14] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, “Overview of the H.264/AVC video coding standard,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.

[15] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[16] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

[17] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, Sep. 2020.

[18] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nat. Methods*, vol. 17, pp. 261–272, Mar. 2020.

[19] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, May/Jun. 2007.

[20] G. Bradski, “The OpenCV library,” *Dr. Dobb’s J. Softw. Tools*, 2000.

[21] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ, USA: Wiley, 2006.

[22] Digital Compression and Coding of Continuous-Tone Still Images—Requirements and Guidelines, ITU-T Recommendation T.81, Sep. 1992.

[23] K. Sayood, *Introduction to Data Compression*, 5th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.